

Phase 7 - Project Executable Files

Project: Rising Waters - Flood Prediction System

Item	Status
Source Code	Yes
README	Yes
requirements.txt	Yes
Dataset	Yes
Model (.pkl)	Yes
Flask App	Yes
Testing	Completed
Deployment	Local
Demo	Available

Technical Architecture

Architecture

Frontend (HTML/CSS/JS) communicates with Flask backend. Flask loads trained ML model and predicts flood risk using weather parameters.

Technologies

Python, Flask, scikit-learn, pandas, NumPy, Joblib, HTML, CSS, JavaScript.

Repository Structure

app.py, model.pkl, dataset.csv, templates/, static/, requirements.txt, README.md

Project Workflow

Step 1: Collect flood dataset.

Step 2: Preprocess missing values and normalize data.

Step 3: Train machine learning model.

Step 4: Evaluate accuracy and save model.

Step 5: Develop Flask backend.

Step 6: Create responsive frontend.

Step 7: Integrate prediction API.

Step 8: Test application.

Step 9: Deploy locally.

Setup & Execution Guide

1. Install Python.
2. `pip install -r requirements.txt`
3. Place dataset and model in project folder.
4. Run **`python app.py`**.
5. Open `http://127.0.0.1:5000`.

The application accepts rainfall, humidity, river level and other inputs, sends them to the trained model and displays flood prediction results.

Conclusion & Deliverables

Submitted Files

Source code, trained model, dataset, README, requirements.txt, screenshots, testing report and executable Flask project.

Future Enhancements

Cloud deployment, live weather API integration, SMS alerts, GIS visualization and mobile application support.

Conclusion

The Rising Waters project is fully documented and organized for academic evaluation. The deliverables include executable source code, documentation, setup instructions and project artifacts.